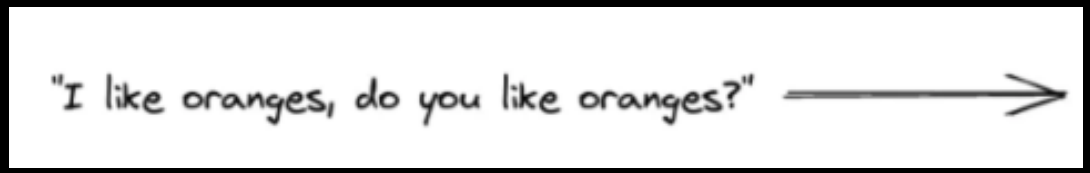
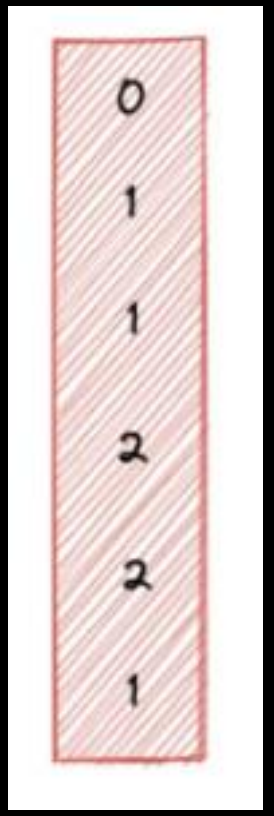
Deep Learning P2-L1
Converting Text into Sequence Data



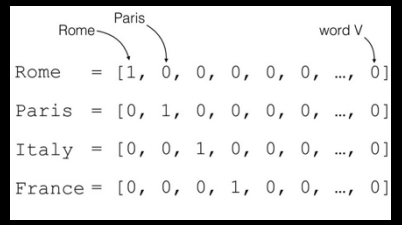


November 2026
Theory for Lab 1

- Tokenization
- Corpus
- Language Statistics: Zipf's Law
- Perplexity

Converting Raw Text into Sequence Data

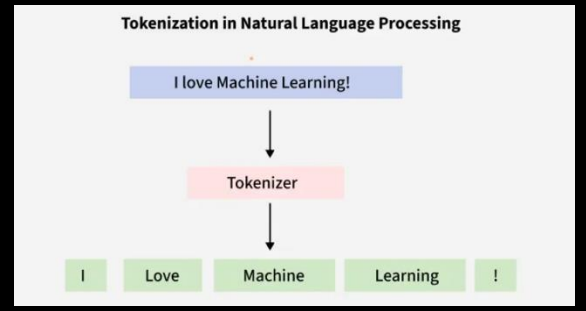




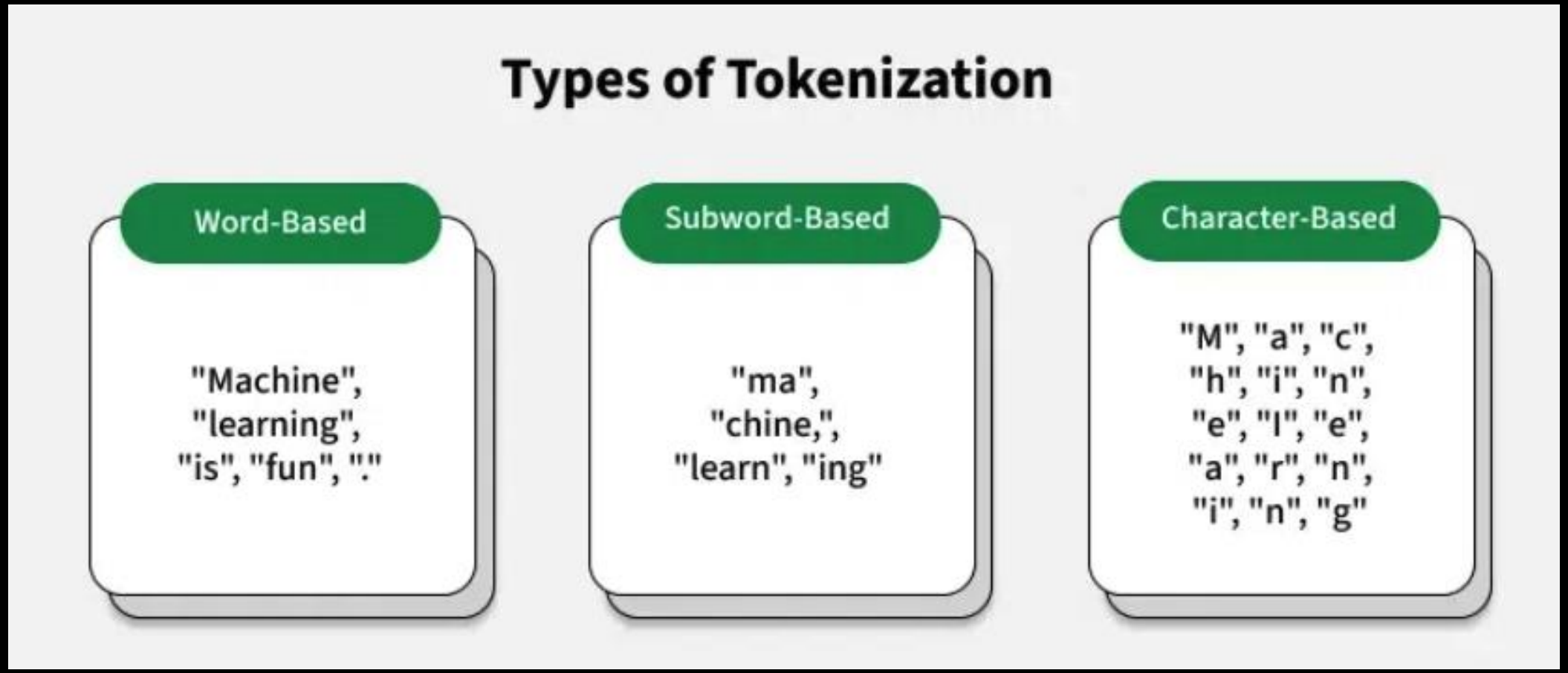
- We need tools for converting raw text into input vectors for the models. Typical preprocessing pipelines do:

1. Load text as strings into memory.
2. Split the strings into tokens (e.g., words or characters).
3. Build a vocabulary dictionary to associate each vocabulary element with a numerical index.
4. Convert the text into sequences of numerical indexes

Tokenization



- Tokens are the atomic (indivisible) units of text.
- What precisely constitutes a token is a design choice (we can use characters or words or something similar).
- For example, we could represent the sentence "Baby needs a new pair of shoes" as a sequence of 7 words, where the set of all words comprise a large vocabulary (typically tens or hundreds of thousands of words).
- Or we would represent the same sentence as a much longer sequence of 30 characters, using a much smaller vocabulary (there are only 256 distinct ASCII characters).


Types of Tokenization
Word-Based
"Machine",
"learning",
"is", "fun", "."
Subword-Based
"ma",
"chine,",
"learn", "ing"
Character-Based
"M", "a", "c",
"h", "i", "n",
"e", "l", "e",
"a", "r", "n",
"i", "n", "g"

Step	Description	Example: "unhappiness"
1. Initial Vocabulary	Start with an initial vocabulary of all individual characters present in the training text. Spaces are often included or handled specifically.	`['u', 'n', 'h', 'a', 'p', 'i', 'e', 's']`
2. Frequency Count	Count the occurrences of all adjacent character or subword pairs in the entire text corpus.	Pair `('n', 'h')` occurs once, `('p', 'p')` occurs once, `('e', 's')` occurs once, etc.
3. Merge Most Frequent	The most frequent pair is identified and merged into a new, single subword token. This new token is added to the vocabulary.	If `('e', 's')` is the most frequent pair in the *entire* corpus, it merges to `'es'`.
4. Repeat	Steps 2 and 3 are repeated iteratively, using the newly formed subwords, until a predefined vocabulary size limit is reached or no more frequent pairs are found.	Next merges might form `'un'`, `'happ'`, then `'unhapp'`, etc.
5. Final Tokenization	For a new word (even an unseen one), the model uses the learned vocabulary and merge rules to break it down into the largest possible subwords found in the vocabulary.	The word "unhappiness" is tokenized into: `['un', 'happ', 'i', 'ness']` (depending on the final vocabulary rules).

Subword tokenization

✅ Handles new words (“unbelievableness” → “un” “believ” “able” “ness”)
✅ Keeps vocabulary small
✅ Balances efficiency + flexibility

Comparison

Feature	Character Tokenization	Word Tokenization	Sub-Word Tokenization
Token Unit	Individual characters	Whole words (usually space/punctuation delimited)	Sub-word units (prefixes, suffixes, roots)
Vocabulary Size	Very small and manageable (e.g., 256 characters)	Very large (e.g., 170,000+ words in English)	Moderate, a balance between the other two
Out-of-Vocabulary (OOV) Words	No OOV issues, as any word is decomposable into known characters	Significant OOV issues; unknown words are marked as generic "unknown" tokens	Handles OOV words well by breaking them into known sub-words
Sequence Length	Very long sequences, increasing computational complexity	Shorter sequences compared to character level	Moderate length, balancing vocabulary size and sequence length
Meaning/Context	Characters usually carry little individual semantic meaning	Words carry strong semantic meaning, aiding in context understanding	Sub-words can carry meaningful linguistic/morphological information

https://medium.com/data-science/word-subword-and-character-based-tokenization-know-the-difference-ea0976b64e17

Dictionary sizes

RAE dictionary : 94.000 Spanish words

Oxford English Dictionary: 301.000 English words

The Time Machine novel we are going to use for experiments: 170580 characters(28 different), 32775 words length (4580 different words)

Average Word length in english is 4.7

The vocabulary size for **GPT-4 is approximately 100,000 tokens, average size is 4.**

Corpus: collection of written or spoken texts on which AIs are trained

Typical Corpus Sizes

Model Type	Corpus Size (approximate)	Examples/Notes
Foundational (General Purpose)	Billions to Trillions of tokens	GPT-3 (175B parameters) was trained on Common Crawl and other sources totaling trillions of tokens. Common Corpus, an open dataset, has about 2 trillion tokens.
Domain-Specific (Pre-training)	Hundreds of billions of tokens	The Stack v2 public code dataset contains 775 billion tokens.
Fine-Tuning	Thousands to millions of tokens	The data depends on the specific task, such as clinical notes or academic papers for specialized applications.

Vocabulary: convert tokens into numbers

- Tokens are our atomic units. We introduce a class for constructing vocabularies, i.e., objects that associate each distinct token value with a unique index. First, we determine the set of unique tokens in our training corpus. We then assign a numerical index to each unique token. Rare vocabulary elements are often dropped for convenience.

- Whenever we encounter a token at training or test time that had not been previously seen, we represent it by a special "<unk>" token, signifying that this is an unknown value.

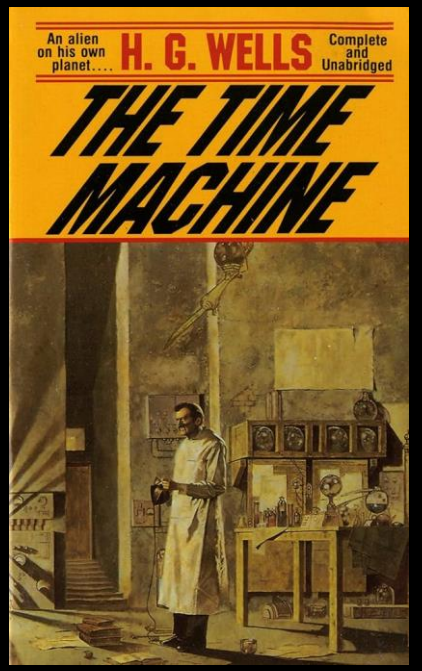
```
vocab = Vocab(tokens)
indices = vocab[tokens[:10]]
print('indices:', indices)
print('words:', vocab.to_tokens(indices))
```

indices: [21, 9, 6, 0, 21, 10, 14, 6, 0, 14]
words: ['t', 'h', 'e', ' ', 't', 'i', 'm', 'e', ' ', 'm']

We download a book, preprocess to eliminate characters we do not consider useful at this time (punctuation symbols, etc.) and then obtain our corpus.

```
len(corpus), len(vocab)
```

```
170580,        28
```



170580 characters in total: length of the corpus

Vocabulary size is 28 (26 letters + 1 space + plus the "<unk>" (unknown) token used internally by the Vocab class for any character not encountered during training.

At character level
the unknown character
first and then the rest
in frequency order.

First the empty space,
then the letter “e”

Total tokens in vocabulary: 28

Index	Token	Frequency
0	<unk>	0
1		29927
2	e	17838
3	t	13515
4	a	11704
5	i	10138
6	n	9917
7	o	9758
8	s	8486
9	h	8257
10	r	7674
11	d	6337
12	l	6146
13	m	4043
14	u	3805
15	c	3424
16	f	3354
17	w	3225
18	g	3075
19	y	2679
20	p	2427
21	b	1897
22	v	1295
23	k	1087
24	x	236
25	z	144
26	j	97
27	q	95

We can also easily explore word statistics

```
words = text.split()


Analyze the frequency of words
vocab = Vocab(words)


vocab.token_freqs[:10]
```

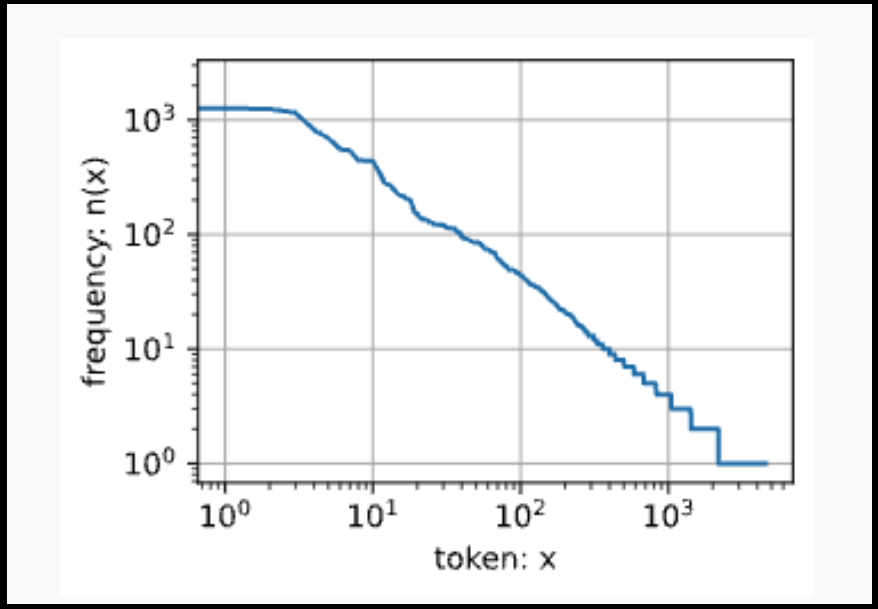
```
[('the', 2261),
 ('i', 1267),
 ('and', 1245),
 ('of', 1155),
 ('a', 816),
 ('to', 695),
 ('was', 552),
 ('in', 541),
 ('that', 443),
 ('my', 440)]
```

Language Statistics: Let's verify Zipf' law

Zipf's law, which states that:

the frequency n_i of the ith most frequent word follows a straight line on a log-log scale.

After dealing with the first few words as exceptions, all the remaining words roughly follow a straight line on a log–log plot.



Zipf's law

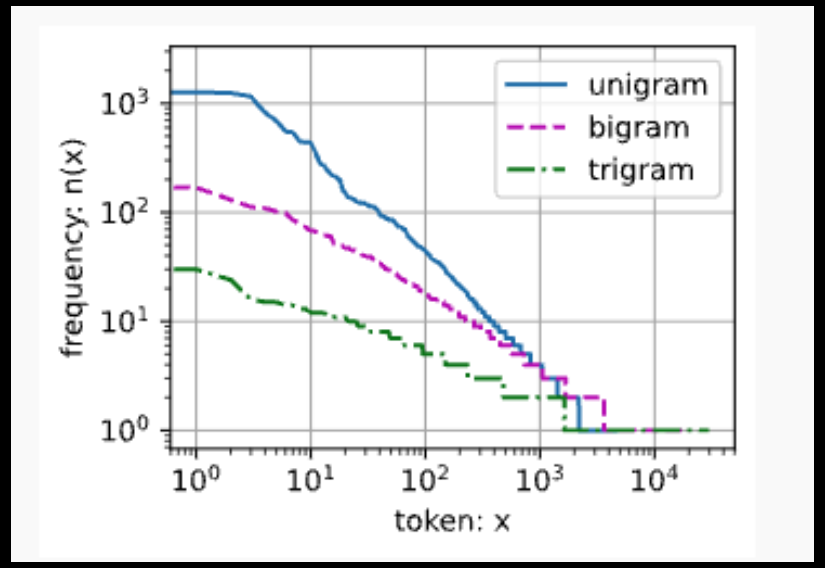
$$n_i \propto \frac{1}{i^\alpha},$$

$$\log n_i = -\alpha \log i + c,$$

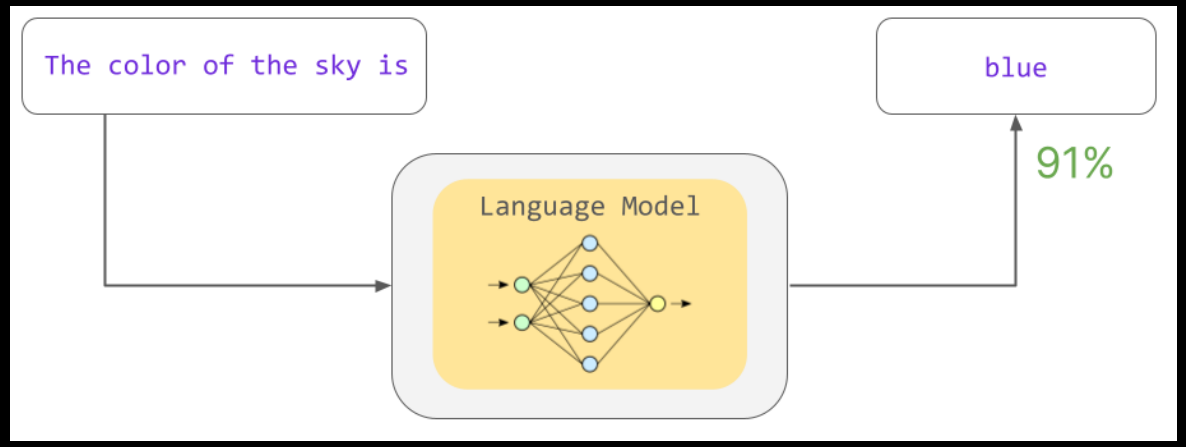
Unigrams-Bigrams-trigrams

- But what about the other word combinations, such as two consecutive words (bigrams), three consecutive words (trigrams), and beyond? Let's see whether the bigram frequency behaves in the same manner as the single word (unigram) frequency.

```
[('of--the', 309),
 ('in--the', 169),
 ('i--had', 130),
 ('i--was', 112),
 ('and--the', 109),
 ('the--time', 102),
 ('it--was', 99),
 ('to--the', 85),
 ('as--i', 78),
 ('of--a', 73)]
```



How to measure the quality of LLM's: **Perplexity**



Next Word prediction is a key component of LLMs

A good language model is able to predict, with high accuracy, the tokens that come next. Consider the following continuations of the phrase "It is raining", as proposed by different language models:

1. "It is raining outside"
2. "It is raining banana tree"
3. "It is raining piouw;kcj pwepoiut"

Perplexity: How to evaluate and compare Language Models

The idea is to measure the uncertainty or the surprise fr the possible choices a language model has for the next Word prediction in a sentence.

The less uncertain the better, as it means that the model predict more accurate the "target" word.

We want a measure that has a mínimum value for a "total" certain model and maximum value for a random one.

Probability and entropy comes to help us, and the measure for this is called PERPLEXITY .

Minimum value = 1

Maximum value = number of words in the vocabulary.

Computing Perplexity from sentence probabilities

Suppose we trained a small language model over an english corpus.

The model is very small and it is only able to predict the probability of the next word in the sentence from a small subset of these words:

"a", *"the"*, *"red"*, *"fox"*, *"dog"*, *"."*

Let's compute the probability of the sentence W, which is "*a red fox.*".

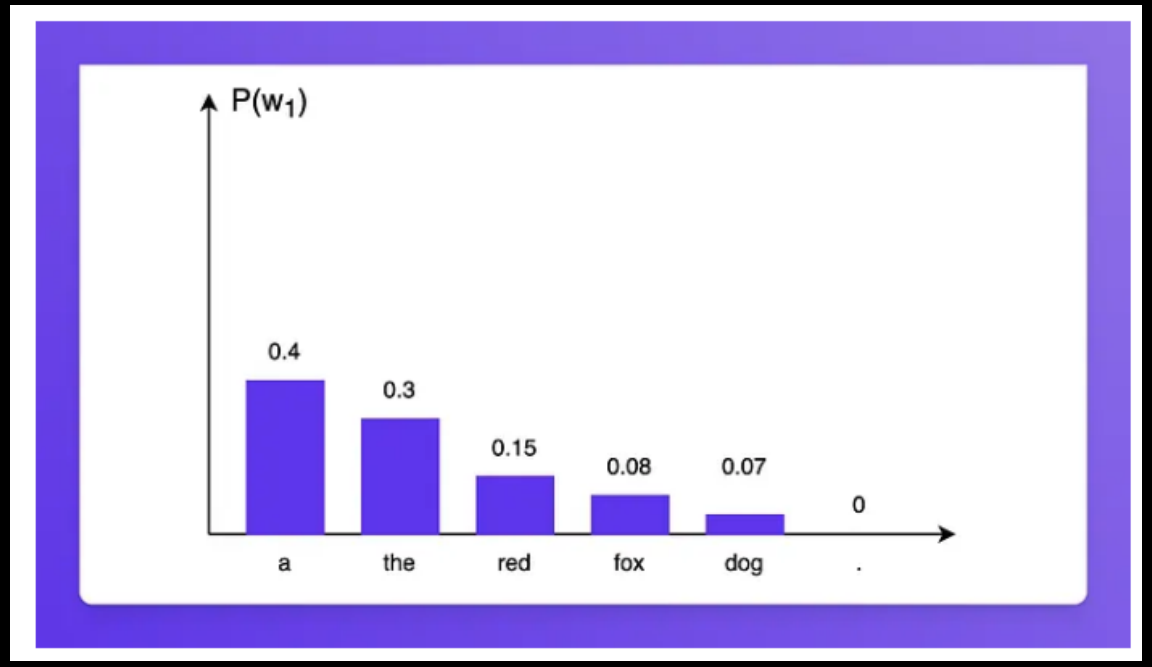
The probability of a generic sentence W, made of the words w_1, w_2, up to w_n, can be expressed as the following:

$$P(W) = P(w_1, w_2, \ldots, w_n)$$

P(“a red fox.”) =
P(“a”) * P(“red” | “a”) * P(“fox” | “a red”) * P(“.” | “a red fox”)

And suppose the following are the probabilities assigned by our language model to a generic **first word** in a sentence:

P(“a”) = 0,4

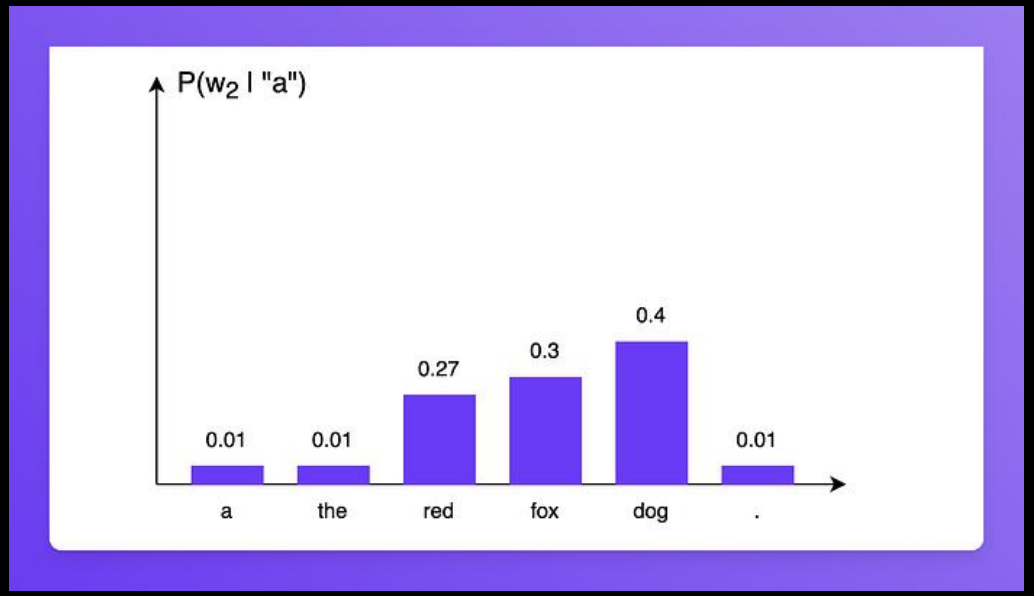


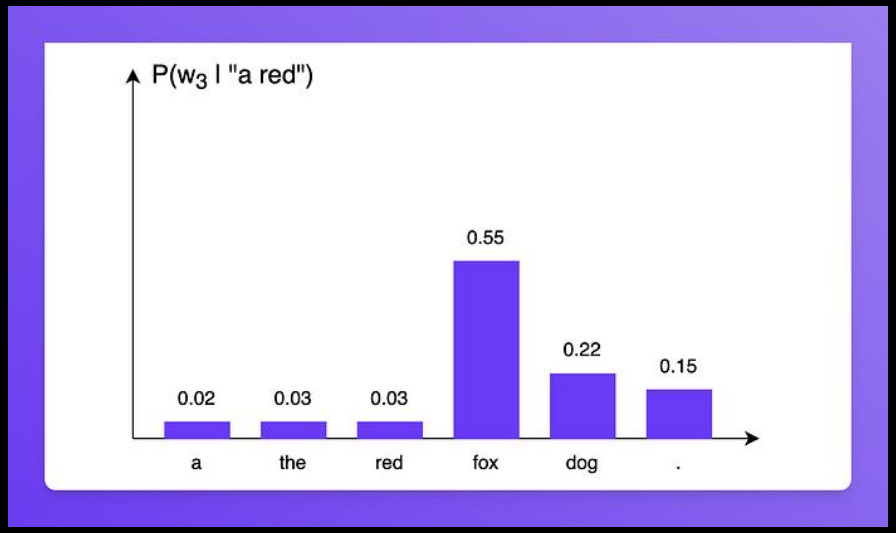
https://medium.com/nlplanet/two-minutes-nlp-perplexity-explained-with-simple-probabilities-6cdc46884584

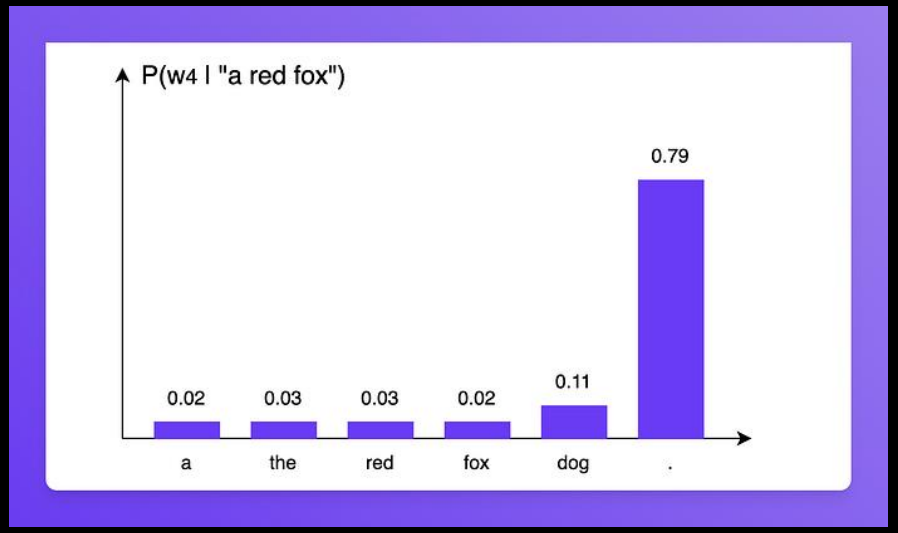
Next, suppose these are the probabilities given by our language model to a generic second word that follows "*a*":

The probability of "*red*" as the second word
in the sentence after "*a*" is:

P("red" | "a") = 0.27







Finally, the probability assigned by our language model to the whole sentence "*a red fox.*" is:

$$P(\text{“a red fox.”}) = P(\text{“a”}) * P(\text{“red”} \mid \text{“a”}) * P(\text{“fox”} \mid \text{“a red”}) * P(\text{“.”} \mid \text{“a red fox”})$$
$$= 0.4 * 0.27 * 0.55 * 0.79$$
$$= 0.0469$$

Since the probability of a sentence is obtained from a product of probabilities, the longer is the sentence the lower will be its probability

Pnorm(W) = P(W) ^ (1 / n)

Pnorm(“a red fox.”) = P(“a red fox”) ^ (1 / 4) = 0.465

Let's call *PP(W)* the perplexity computed over the sentence *W*.

And define it as the reciprocal of the normalized probability.

$$PP(W) = 1 / Pnorm(W) = 1 / 0.465 = 2,15$$

The lower the perplexity over a well-written sentence
the better is the language model.

Let's try computing the perplexity with a second language model that assigns **equal probability** to each word at each prediction (a random model).

Since the language models can predict six words only, the probability of each word will be 1/6.

P("a red fox.") = (1/6) ^ 4 = 0.00077

Pnorm("a red fox.") = P("a red fox.") ^ (1/4) =((1/6)^4)^1/4

PP("a red fox") = 1 / Pnorm("a red fox.") = 6

$$\text{Perplexity}(W) = \sqrt[N]{\frac{1}{P(w_1, w_2, \ldots, w_N)}} =$$

$$\exp\left(-\frac{1}{n}\sum_{t=1}^{n}\log P(x_t \mid x_{t-1}, \ldots, x_1)\right).$$

exp(Cross-entropy)

Perplexity is defined as the reciprocal of the geometric mean of the number of real choices that we have when deciding which token to pick next. Let's look at a number of cases:

• In the best case scenario, the model always perfectly estimates the probability of the target token as 1. In this case the perplexity of the model is 1.

• At the baseline, the model predicts a uniform distribution over all the available tokens of the vocabulary. In this case, the perplexity equals the number of unique tokens of the vocabulary

Interpretation

- **Lower is Better:** A lower perplexity value indicates the model is more confident in its predictions and a better language model.
- **Intuitive Meaning:** Perplexity can be interpreted as the effective number of choices the model has for the next word. A perplexity of 10 suggests the model is as uncertain as if it were choosing from 10 equally probable options for the next word.

📊 Perplexity Scores of Popular LLMs

While exact values vary by dataset and evaluation method, here are some typical estimates:

- **GPT-2 (small)**: ~35 on WikiText-2
- **GPT-2 (large)**: ~20 on WikiText-2
- **GPT-3**: Estimated ~20–30 on various corpora (OpenAI hasn't published exact scores)
- **LLaMA (7B)**: ~10–15 on standard benchmarks like Penn Treebank or WikiText

Assessing performance of Large Language Models

2018-2022: Lower perplexity strongly correlated with better language models.

2023-2026: Lower perplexity remained necessary but became insufficient to explain capability gains.

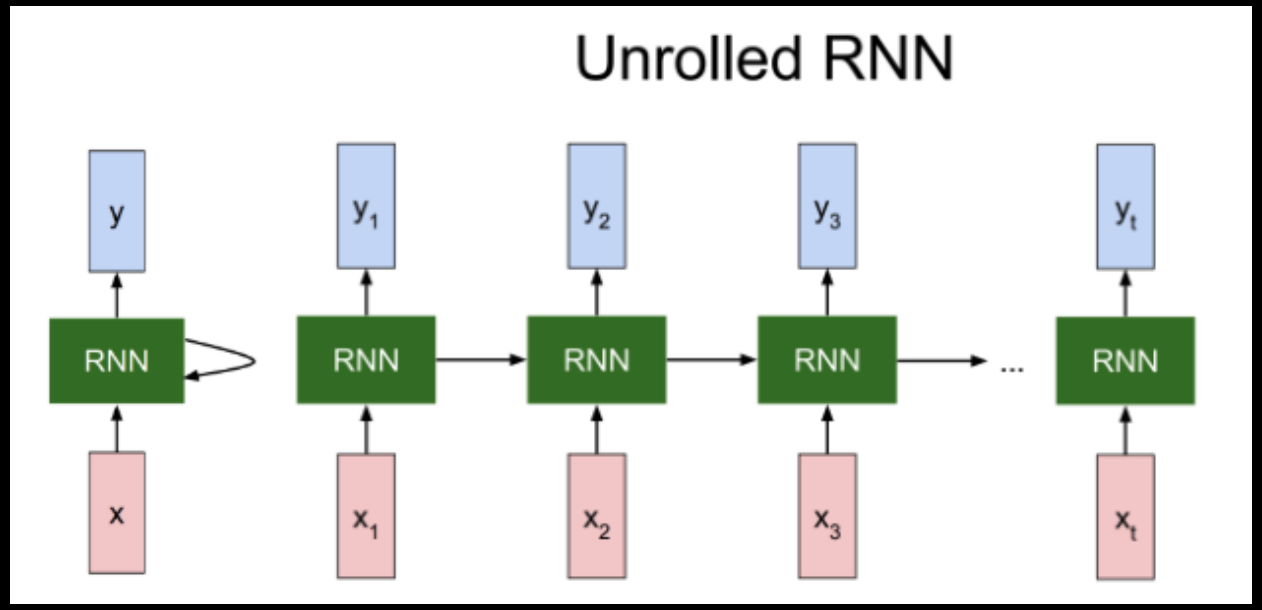
Perplexity measures how "surprised" a model is by the next token. Mathematically,

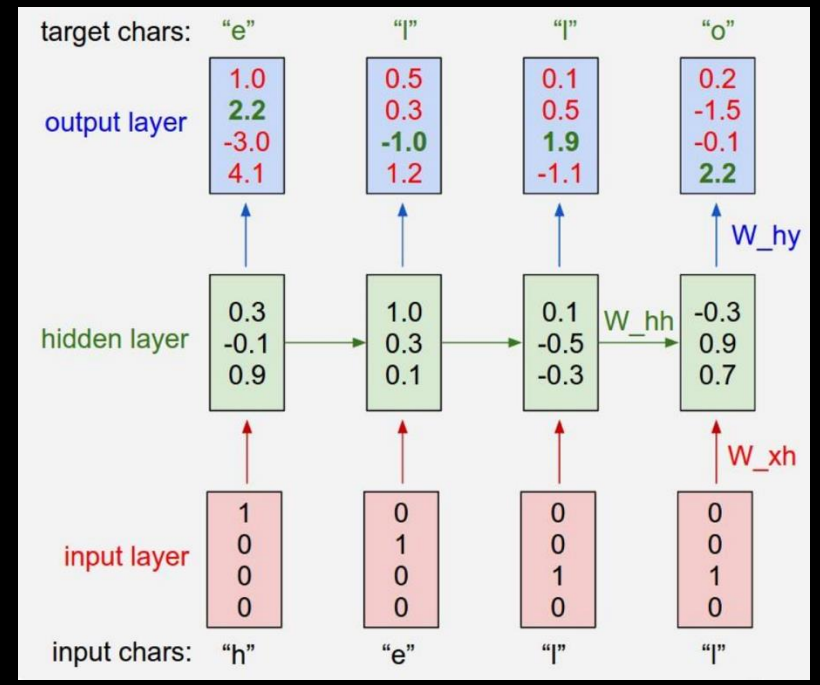
$$PPL = \exp\left(-\frac{1}{N}\sum_{i=1}^{N}\log p(x_i)\right)$$

Lower is better. However, modern LLMs are increasingly evaluated on:

- reasoning benchmarks (GPQA, MATH, AIME),
- agentic tasks,
- coding performance
- factuality,
- long-context retrieval.

In Lab 2 we will see how to train a RNN, now the task in Lab 1 is to work with sequences and metrics





https://cs231n.github.io/rnn/#:~:text=In%20the%20most%20simplest%20form,ht%20at%20timestep%20t.

Text input for RNN: one-hot encoding

We have an index for the tokens and we represent them using a vector whose length is given by the size of the vocabulary , where all entries are set to 0, except for the entry corresponding to our token, which is set to 1 . For example, if the vocabulary had four elements, then the one-hot vectors corresponding to indices 0 to 3 would be the following:

Consider a simple vocabulary: `{"h": 0, "e": 1, "l": 2, "o": 3}` (vocabulary size = 4).

The sequence "hello" would be processed as follows:

Character	Integer Encoding	One-Hot Encoding Vector
h	0	`[1, 0, 0, 0]`
e	1	`[0, 1, 0, 0]`
l	2	`[0, 0, 1, 0]`
l	2	`[0, 0, 1, 0]`
o	3	`[0, 0, 0, 1]`

Evolución de las representaciones de texto

Etapa	Representación	Captura	Limitación
One-Hot	`[0,1,0,...]`	Identidad	Sin semántica
Frecuencias	BoW, TF-IDF	Importancia estadística	Sin significado
Fixed Emb.	Word2Vec, GloVe	Semántica general	Un vector por palabra
Context. Emb.	BERT, GPT, Llama	Significado contextual	Mayor complejidad

Other encodings/embeddings: BoW

Bag-of-Words (BoW) encoding is generally **not used as the direct input method for a standard recurrent neural network (RNN) for sequence modeling**. This is because the fundamental characteristic of BoW is that it **discards the order** of items (words or characters), while RNNs are specifically designed to process and leverage the *sequential* order and context of data.

Input Text: "cat in the hat"
Tokens (Characters): ['c', 'a', 't', ' ', 'i', 'n', ' ', 't', 'h', 'e', ' ', 'h', 'a', 't']

Unique Characters (Vocabulary): ['c', 'a', 't', ' ', 'i', 'n', 'h', 'e']
index: ['0', '1', '2', '3', '4', '5', '6', '7']
Vocabulary Size: 8

Initialize the BoW vector with zeros a vector of lenght equal to vocabulary size: [0, 0, 0, 0, 0, 0, 0, 0]

Iterate the input "cat in the hat" through tokens and count:

- 'c': appears 1 time.
- 'a': appears 2 times.
- 't': appears 2 times.
- ' ': appears 3 times.
- 'i': appears 1 time.
- 'n': appears 1 time.
- 'h': appears 2 times.
- 'e': appears 1 time.

The final Bag-of-Words encoded vector for the text "cat in the hat" is:

'c'	'a'	't'	' '	'i'	'n'	'h'	'e'
1	2	2	3	1	1	2	1

The final Bag-of-Words encoded vector for the text "cat in the hat" is:

This vector effectively represents the *content* of the text (which characters are present and how often) but completely ignores the *order* in which the characters appeared. For example, the text "the in cat hat" would produce the exact same BoW vector.

TF-IDF: Term Frequency-Inverse Document Frequency

TF-IDF highlights words that are most representative and discriminative of a document's content, making it a widely used method in information retrieval, text mining, and search engines.

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D)$$

$$\text{TF}(t, d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of words in document } d}$$

$$IDF(t, D) = \log\left(\frac{\text{Total Number of Documents in } D}{\text{Number of Documents Containing } t}\right)$$

TF-IDF: Term Frequency-Inverse Document Frequency

Consider a small corpus of **two** documents:

- **Document 1:** "The cat sat on the mat"
- **Document 2:** "The dog chased the cat"

Step 1: Calculate Term Frequency (TF)

$$\mathrm{TF}(t, d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of words in document } d}$$

Term	Document 1 (6 words)	TF in Doc 1	Document 2 (5 words)	TF in Doc 2
the	2 times	2/6 ≈ 0.333	2 times	2/5 = 0.4
cat	1 time	1/6 ≈ 0.167	1 time	1/5 = 0.2
sat	1 time	1/6 ≈ 0.167	0 times	0/5 = 0.0
on	1 time	1/6 ≈ 0.167	0 times	0/5 = 0.0
mat	1 time	1/6 ≈ 0.167	0 times	0/5 = 0.0
dog	0 times	0/6 = 0.0	1 time	1/5 = 0.2
chased	0 times	0/6 = 0.0	1 time	1/5 = 0.2

Step 2: Inverse Domain Frequency (IDF)

$$IDF(t, D) = \log\left(\frac{\text{Total Number of Documents in } D}{\text{Number of Documents Containing } t}\right)$$

Term	Documents Containing Term	IDF Calculation (log base 10)	IDF Score
the	2 (Doc 1, Doc 2)	log(2/2) = log(1)	0.0
cat	2 (Doc 1, Doc 2)	log(2/2) = log(1)	0.0
sat	1 (Doc 1)	log(2/1) = log(2)	≈ 0.301
on	1 (Doc 1)	log(2/1) = log(2)	≈ 0.301
mat	1 (Doc 1)	log(2/1) = log(2)	≈ 0.301
dog	1 (Doc 2)	log(2/1) = log(2)	≈ 0.301
chased	1 (Doc 2)	log(2/1) = log(2)	≈ 0.301

Step 3: Compute TF-IDF

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D)$$

Term	TF-IDF in Doc 1 (TF * IDF)	TF-IDF in Doc 2 (TF * IDF)
the	0.333 * 0.0 = 0.0	0.4 * 0.0 = 0.0
cat	0.167 * 0.0 = 0.0	0.2 * 0.0 = 0.0
sat	0.167 * 0.301 ≈ 0.050	0.0 * 0.301 = 0.0
on	0.167 * 0.301 ≈ 0.050	0.0 * 0.301 = 0.0
mat	0.167 * 0.301 ≈ 0.050	0.0 * 0.301 = 0.0
dog	0.0 * 0.301 = 0.0	0.2 * 0.301 ≈ 0.060
chased	0.0 * 0.301 = 0.0	0.2 * 0.301 ≈ 0.060

"The cat sat on the mat":

[0.0, 0.0, 0.05, 0.05, 0.05, 0.0, 0.0]

Index: [the, cat, sat, on, mat, dog, chased]

"The dog chased the cat"

[0.0, 0.0, 0.0, 0.0, 0.0, 0.06, 0.06]

Index: [the, cat, sat, on, mat, dog, chased]

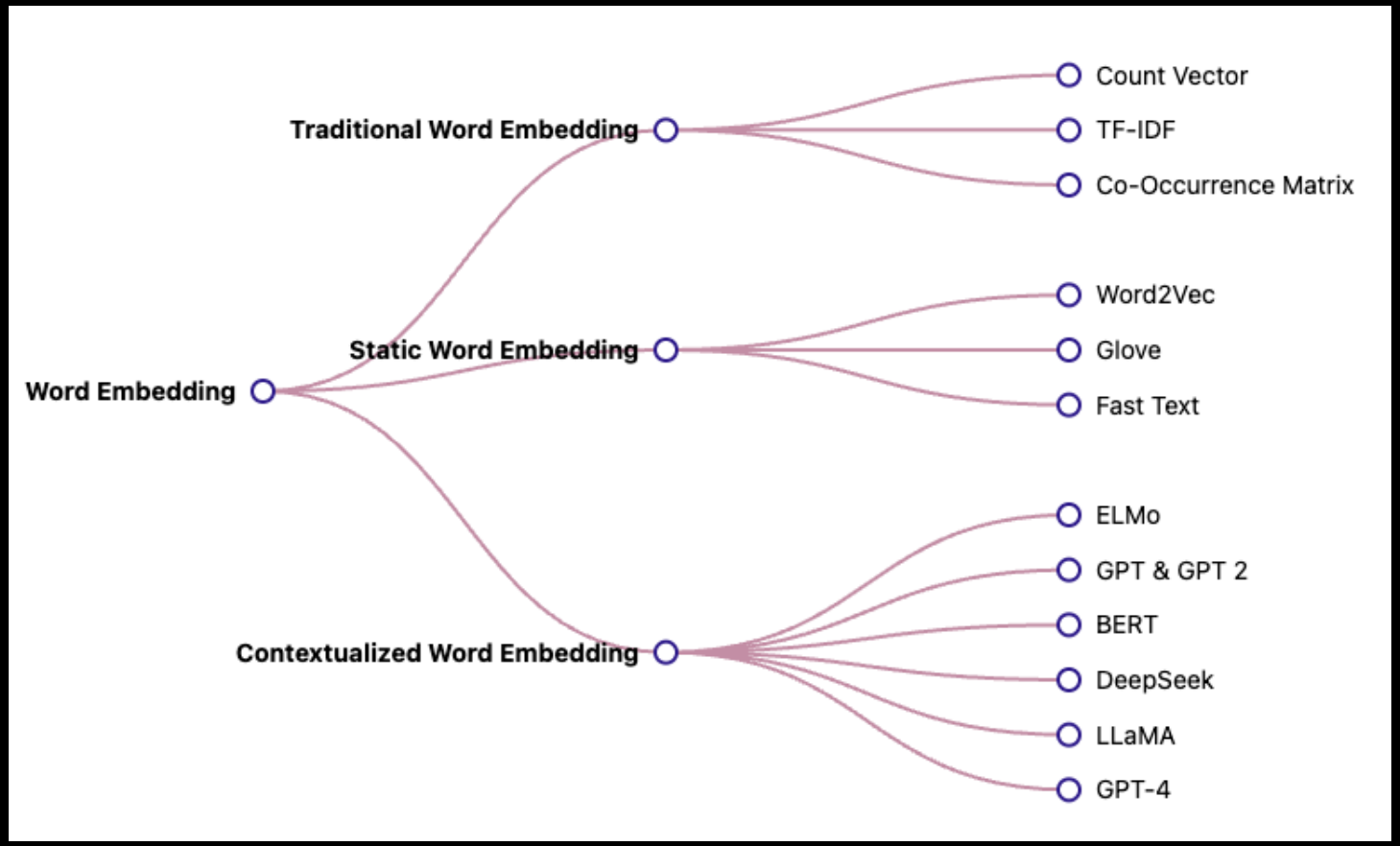
TF-IDF: Term Frequency-Inverse Document Frequency

- **Simplicity and Speed:** TF-IDF is easy to implement, doesn't require training on labeled data, and is computationally efficient.

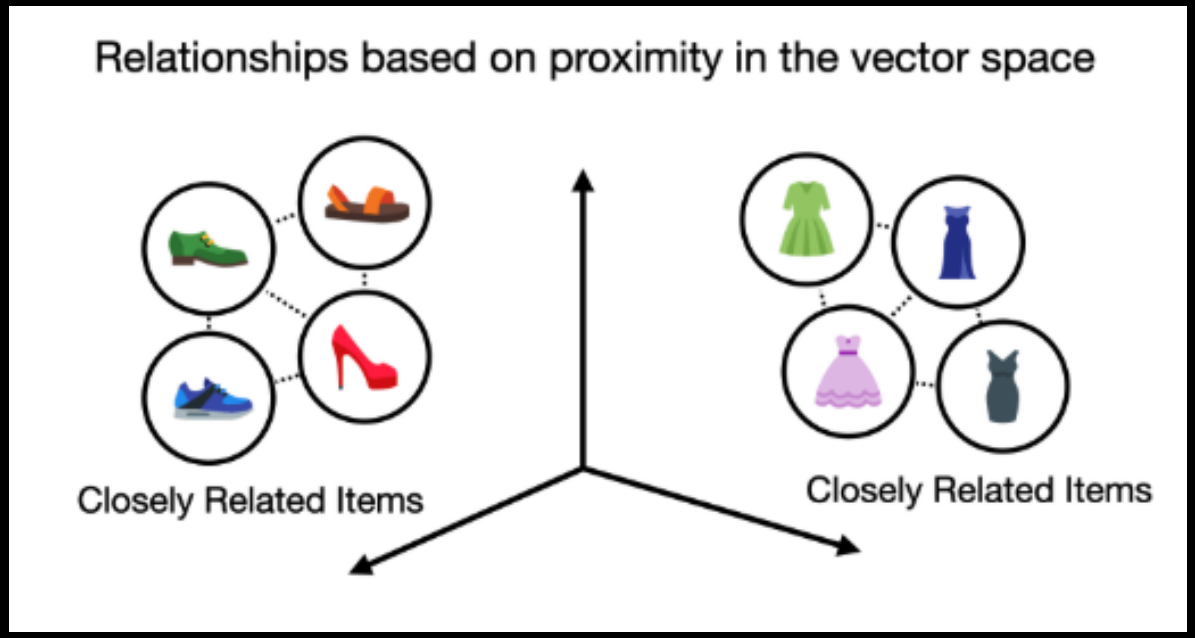
- **Contextual Differences:** In some specific scenarios with limited data or for simpler tasks, studies have shown that TF-IDF can even outperform word embeddings.

TF-IDF was a successful, long-standing method that preceded modern, deep-learning-based word embeddings, which address some of its key limitations (like a lack of semantic understanding).

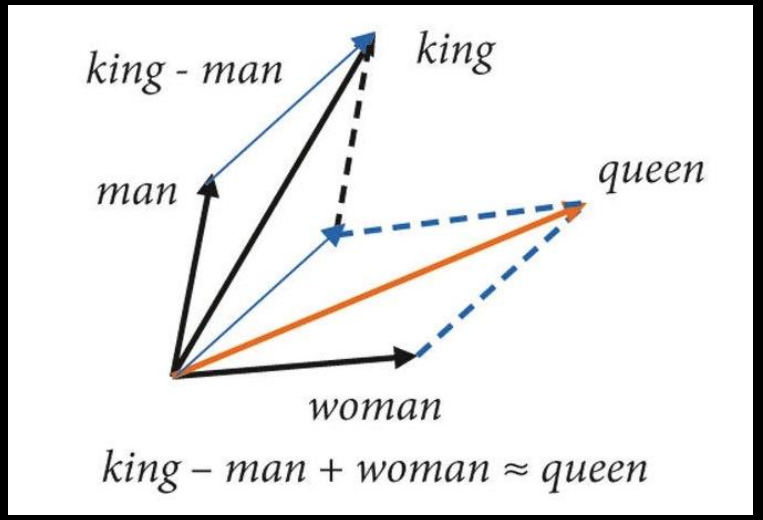
Embeddings: mapping complex, high-dimensional data (such as words, images, or entire documents) into a **vector space** specifically designed to capture semantic meaning and relationship



Word embeddings: Relaciones semánticas en espacios vectoriales



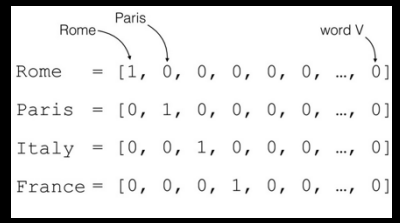
Products with similar meanings tend to be closer in this space



Embedding space learns a gender-related direction from patterns.

Laboratory 1
Converting Text into Sequence Data

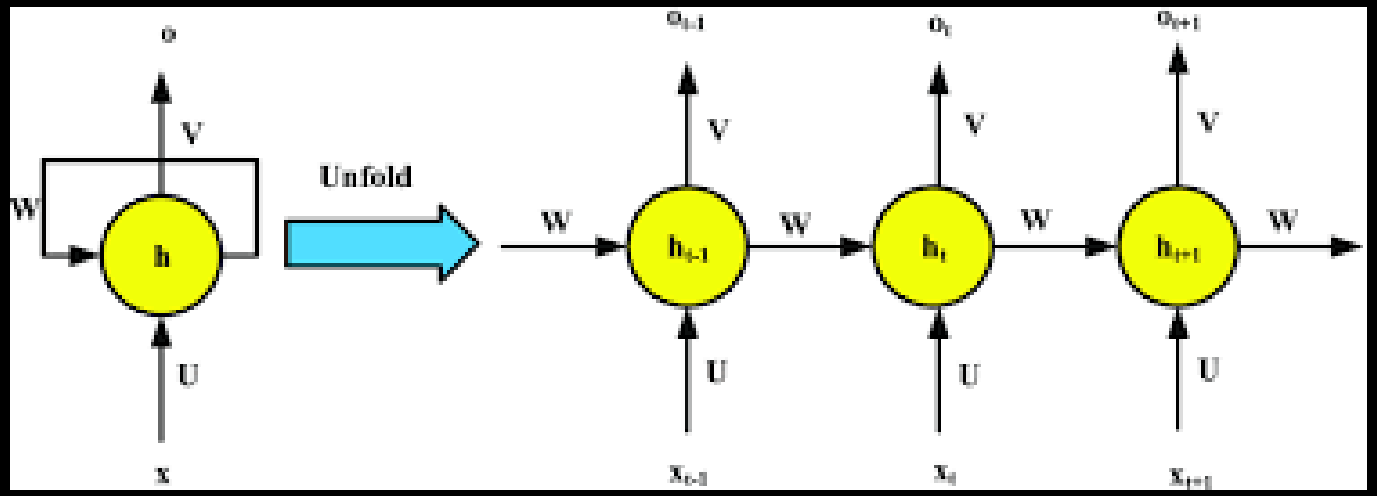




- Read a text file
- Pre-process it
- Tokenize
- Analyze Language statistics

Laboratory 2

Train a Recurrent Neural Network



P2-Lab 1: Tareas

1) Ejecutar el cuaderno provisto en el Campus Virtual (P2-Lab1.ipynb) y familiarizarse con las tareas, código y librerías utilizadas. El cuaderno incluye descarga del libro “The Time Machine”, tokenización en caracteres, , análisis de frecuencia de palabras, representación gráfica en escala log-log de esta frecuencia para unigramas, bigramas y trigramas.

2) Realizar un ajuste lineal para calcular el exponente de la ley de Zipf con los datos del conjunto “The Time Machine” para unigramas, bigramas y trigramas (palabras), eliminando los primeros valores con frecuencia muy alta y los últimos valores con frecuencia 1.

3) Repetir todos los análisis previos con el archivo provisto en español del libro “Juvenilia.txt” disponible en el campus virtual.

4) Extraer conclusiones en particular y comparativamente entre los idiomas español e inglés, considerando también que hemos utilizado un corpus limitado.

5) Revisar las codificaciones One Hot Encoding y TF-IDF y la métrica de Perplejidad.

Resources

https://pandalab.me/archives/converting_raw_text_into_sequence_data

https://gdevakumar.medium.com/how-do-llms-process-text-data-bpe-and-embedding-part-2-ec8f9b0a7bce

https://cset.georgetown.edu/article/the-surprising-power-of-next-word-prediction-large-language-models-explained-part-1/

https://medium.com/nlplanet/two-minutes-nlp-perplexity-explained-with-simple-probabilities-6cdc46884584

https://ompramod.medium.com/exploring-n-grams-the-building-blocks-of-natural-language-understanding-3e89087412e4

https://ebetica.github.io/pytorch-Deep-Learning/en/week06/06-3/#:~:text=With%20an%20easy%20level%20of,to%20train%20than%20the%20LSTM.

https://colab.research.google.com/github/agungsantoso/deep-learning-v2-pytorch/blob/master/sentiment-rnn/Sentiment_RNN_Exercise.ipynb#scrollTo=iyt2In2xqd5Z